

Czak Noris: agent dwuwarstwowy z selektorem trybów uczonym metodą Double Q-Learning

Kamil Baran Marcin Wojnar
Hubert Kozierkiewicz

Raport z projektu — Uczenie ze Wzmocnieniem (GUPB), runda 2

Lipiec 2026

1 Architektura: tryby zamiast akcji

Czak Noris gra w GUPB — turową grę battle-royale na siatce 2D z obserwacją częściową (stożkowe pole widzenia, 8 HP na start, bronie o różnej geometrii ataku, mikstury +5 HP, mgła zaciskająca się wokół menhira). Punktacja zależy wyłącznie od kolejności śmierci i jest przyznawana na podstawie lekko zmodyfikowanej funkcji Fibonacciego: przy 12 graczach zwycięzca dostaje 41 pkt, drugi 30, a różnice między dalszymi kolejnymi miejscami są mniejsze. W rundzie 2 gra toczy się na 10 nieznanych z góry mapach generowanych skryptem `arena_generator.py` (teren Perlina, budynki z bronią w środku, losowa pozycja menhira).

Kluczowa decyzja projektowa: **bot nie podejmuje decyzji na poziomie akcji elementarnych**. Decyzja strategiczna — „w jakim trybie teraz gram” — zapada co $K = 8$ tur (lub wcześniej przy zdarzeniu: pojawienie się wroga, utrata ≥ 2 HP, dezaktualizacja celu), a wykonanie trybu należy w całości do deterministycznych heurystyk. Selektor trybu jest wymienny na selektor kaskadowy oraz selektor uczony.

1.1 Tryby

Bot rozróżnia pięć trybów zachowania: **hunt** — dobór celu punktacją (dystans, HP wroga, przewaga broni, czy wróg na nas patrzy) i podejście na pozycję strzału z omijaniem pól rażenia pozostałych przeciwników; **loot** — marsz po miksturę (priorytet przy zranieniu) lub broń wyższego tieru; **explore** — eksploracja nieznanych obszarów, w praktyce głównie poszukiwanie menhira; **menhir** — marsz do menhira i camping (obróć skanujący resetujący karę bezczynności, okazyjne ataki, ładowanie łuku); **flee** — wybór kroku minimalizującego zagrożenie: poza pola rażenia wszystkich widocznych wrogów, dalej od nich, w stronę bezpiecznego celu; osaczony bot odwraca się i walczy.

1.2 Odruchy

Ponad trybami działa cienka warstwa odruchów sprawdzana co turę: zejście z pola ognia/mgły, atak gdy wróg stoi w polu rażenia naszej broni, ładowanie łuku gdy nikogo nie widać. Pola rażenia wyznaczamy bezpośrednio klasami broni silnika (`weapons.cut_positions`), więc geometria (linia miecza blokowana przez postaci, wachlarz topora, przekątne amuletu, dwutaktowy łuk) jest zawsze zgodna z rzeczywistością gry.

1.3 Warstwa wykonawcza

Trzy elementy robią największą różnicę względem naszego bota z rundy 1:

1. **pełna mapa od pierwszej tury** — w `reset()` wczytujemy plik areny, więc znamy topologię i początkowe pozycje broni zanim cokolwiek zobaczymy; obserwacjami uzupełniamy tylko elementy dynamiczne (gracze, mikstury, mgła, zabrany loot);

2. **dwupoziomowy BFS** — najpierw bezpieczna ścieżka (omijająca pola rażenia wrogów, mgłę i ogień), a gdy taka nie istnieje — dowolna; postaci widoczne na mapie są ruchomymi przeszkodami;
3. **model mgły** — replikujemy harmonogram silnika (promień maleje co $2 \cdot$ liczba żywych epizodów, z korektą z obserwacji), co daje ciągły margines bezpieczeństwa „promień — dystans do menhira” zamiast sztywnego progu czasowego.

1.4 Selektor: kaskada priorytetów

Wariant grający w turnieju to uporządkowana lista warunków, bazująca na doświadczeniach z rundy 1: ucieczka przy niskim HP i niekorzystnym starciu $\rightarrow$ mikstura przy zranieniu $\rightarrow$ ulepszenie słabej broni $\rightarrow$ menhir przy mgłę w bliskiej odległości $\rightarrow$ polowanie, kiedy mamy przewagę $\rightarrow$ menhir $\rightarrow$ eksploracja. Kolejność szczebli jest strategią: najpierw przeżycie, potem zasoby, na końcu agresja.

2 Uczenie selektora

2.1 Double Q-Learning nad opcjami (SMDP)

Tryby są opcjami w sensie SMDP: decyzja obowiązuje przez $\tau \leq K$ tur, więc uczenie odbywa się na poziomie segmentów, nie pojedynczych kroków.

Stan to krotka sześciu dyskretnych cech (1296 kombinacji): przedział HP (3), klasa broni (3), zagrożenie od najbliższego wroga (4: brak / dalekie / bliskie korzystne / bliskie niekorzystne), pilność mgły z marginesu promień–dystans (4), znany loot (3), liczba żywych (3). Cechy są niezależne od mapy, co pozwala na transfer na areny nieznane w treningu.

Nagroda jest celowo minimalna: w trakcie segmentu $r_t = 0,5 \cdot \Delta\text{HP}$ na turę, a nagrodą terminalną jest surowy wynik turniejowy (1–41). Sygnał uczący jest więc zdominowany przez faktyczny cel optymalizacji, bez nagród zastępczych.

Aktualizacja (segment o długości τ , zdyskontowana suma nagród $R = \sum_{i=0}^{\tau-1} \gamma^i r_{t+i}$): losowo aktualizujemy jedną z dwóch tablic,

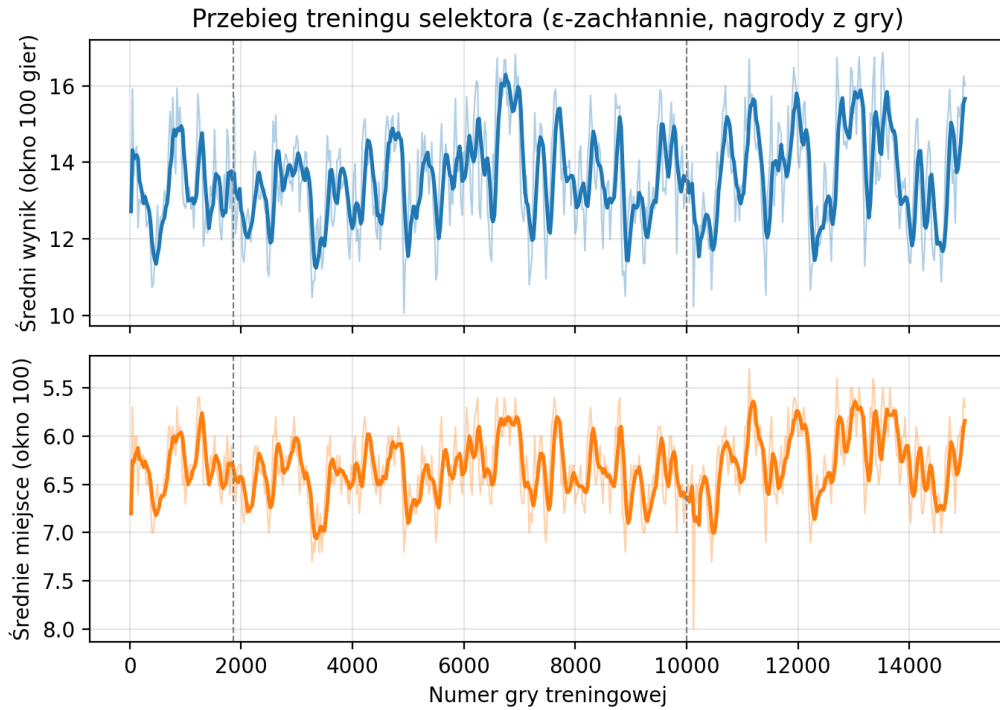
$$Q_1(s, m) \leftarrow Q_1(s, m) + \alpha \left[R + \gamma^\tau Q_2(s', \arg \max_{m'} Q_1(s', m')) - Q_1(s, m) \right], \quad (1)$$

symetrycznie dla Q_2 ; na końcu gry bootstrap zastępowany jest wynikiem terminalnym. Podwójna tablica eliminuje dodatni błąd przeszacowania operatora $\max$ klasycznego Q-Learningu. W turnieju polityka byłaby zamrożona i zachłanna względem $Q_1 + Q_2$, z powrotem do kaskady dla stanów nieodwiedzonych w treningu.

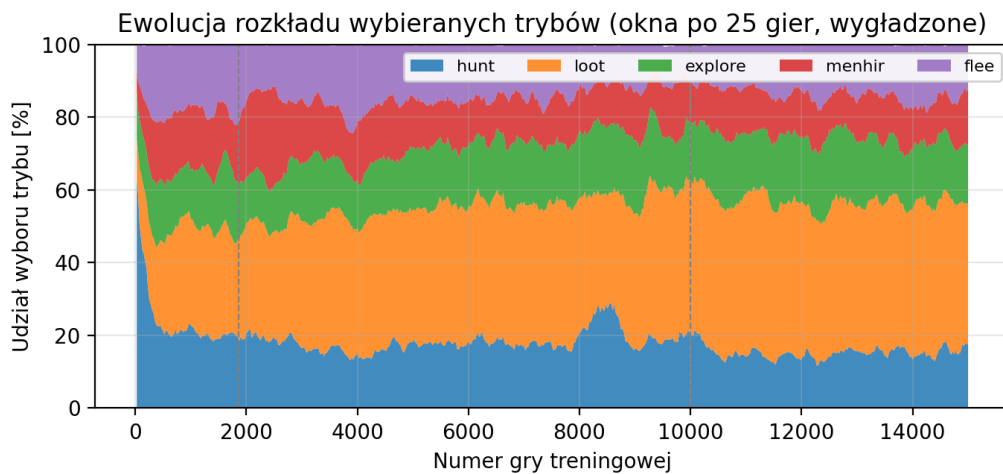
Hiperparametry: $\alpha = 0,1$; $\gamma = 0,99/\text{turę}$; ε liniowo $0,30 \rightarrow 0,05$ przez pierwsze 5000 gier.

2.2 Trening

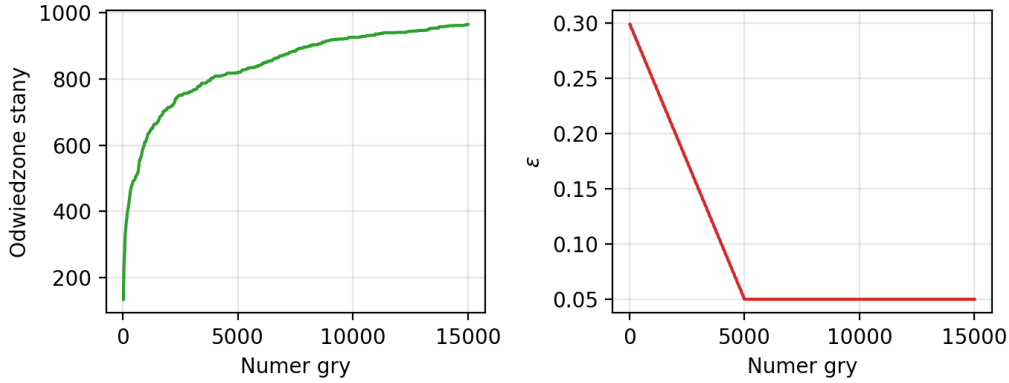
Trening prowadzony jest w środowisku uruchamiającym silnik gry, przeciw pełnej dwunastce kontrolerów z poprzedniej rundy — w tym botom, które same uczą się online między grami. Pula treningowa: 26 aren wygenerowanych tym samym skryptem co areny turniejowe (20 map 24×24 i 6 większych, 28–42), mapa losowana per gra. Łącznie **15 000 gier** w trzech segmentach: 0–1,9 tys. z pełną stawką, 1,9–10 tys. z podmienionym najwolniejszym botem (przepustowość $\sim 0,8$ s/grę), 10–15 tys. ponownie z pełną stawką przy $\varepsilon = 0,05$. Tablica zapisywana atomowo co 50 gier, trening w pełni wznawialny.



Rysunek 1: Kroczący (okno 100 gier) średni wynik i średnie miejsce podczas treningu; linie przerywane — zmiany stawki treningowej. Polityka gra tu z szumem ϵ , więc wartości zaniżają jakość polityki zachłannej.



Rysunek 2: Ewolucja rozkładu wybieranych trybów. Tablica odchodzi od początkowo częstego **hunt** na rzecz **loot** ($\sim 40\%$ — wczesne uzbrajanie się z budynków) i sytuacyjnego **flee**; rozkład pozostaje zróżnicowany.



Rysunek 3: Liczba odwiedzonych stanów (nasycenie ~ 960 z 1296 możliwych) oraz harmonogram ε .

2.3 Podejście alternatywne: PPO na akcjach elementarnych

Równolegle sprawdziliśmy podejście o przeciwnym poziomie abstrakcji: PPO (Stable-Baselines3, polityka aktor–krytyk MLP) wybierające bezpośrednio jedną z 8 akcji elementarnych silnika, z obserwacją 15 znormalizowanych cech (HP, flaga i wektor do menhira, flaga i wektor do najbliższego wroga, „radar” czterech sąsiednich pól: przechodniość i mgła). Trening: 500 tys. kroków środowiskowych ($n_{\text{steps}} = 2048$, batch 64, 10 epok/aktualizację, $\gamma = 0,99$, GAE $\lambda = 0,95$, $\text{lr} = 3 \cdot 10^{-4}$, $\text{ent_coef} = 0$) w osobnym środowisku. Ten wariant musi nauczyć się nawigacji, geometrii broni i strategii jednocześnie — mierzy, ile kosztuje rezygnacja z hierarchii.

3 Wyniki

Wszystkie warianty ocenialiśmy tym samym protokołem: **10 map hold-out** (24×24 , z tego samego generatora, nieużywane w treningu — odpowiednik nieznanymi map rundy 2), pełna stawka turniejowa, polityki zamrożone, a przed każdą grą ustalane ziarno losowe ($\text{seed} = 10000 + i$), przez co warianty grają na identycznej sekwencji map, spawnów i menhirów. Zbiorcze wyniki przedstawia Tabela 1 i Rysunek 4.

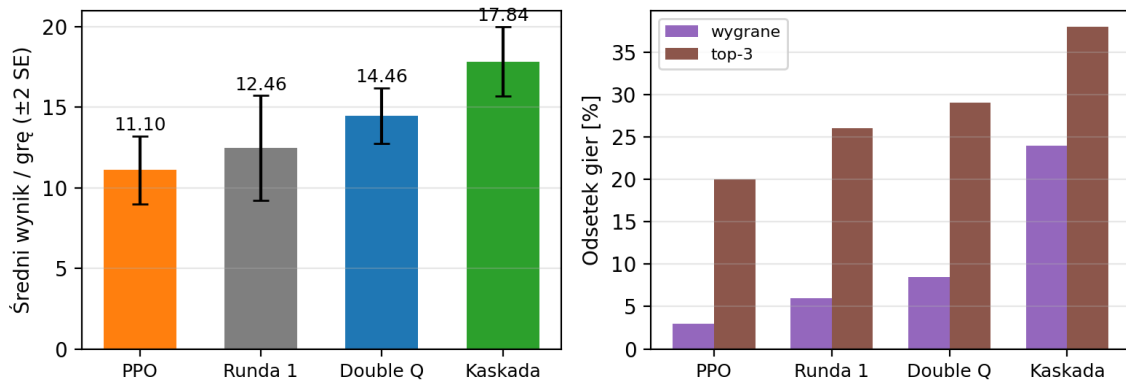
Wariant	Gier	Śr. wynik $\pm$ SE	Śr. miejsce	Wygrane	Top-3
PPO (akcje elementarne)	100	$11,10 \pm 1,06$	6,98	3%	20%
Bot z rundy 1	50	$12,46 \pm 1,63$	6,54	6%	26%
Selektor Double Q	200	$14,46 \pm 0,86$	5,92	8,5%	29%
Selektor kaskadowy	200	$17,84 \pm 1,08$	5,45	24%	38%

Tabela 1: Ewaluacja na mapach hold-out (pełna stawka turniejowa, sparowane ziarna).

Nowa warstwa wykonawcza odpowiada za główny skok jakości (+43% względem rundy 1 przy kaskadzie). Wyuczony selektor pobija bota z rundy 1 (+16%), ale przegrywa z kaskadą o 3,4 pkt/grę, a wyniki czterech wariantów rosną monotonicznie z poziomem abstrakcji, na którym operuje decyzja. W symulacji pełnego turnieju (200 gier przez oficjalny runner) wariant kaskadowy zajął **2. miejsce w stawce** (3334 pkt, za MCTS-owym liderem ligi), wariant Double Q — 6. (2343 pkt). Dlatego do gry w rundzie 2 wybraliśmy wariant kaskadowy. Pełna infrastruktura uczenia (tablica, trening, ewaluacja) pozostaje w repozytorium (`python -m gupb.controller.czak_noris.train / eval`).

Różnica między selektorami bierze się niemal w całości z gier o zwycięstwo: Double Q wygrywa 8,5% gier wobec 24% kaskady, przy zbliżonym odsetku top-3. Polityka uczona na rzadkiej

Ewaluacja na 10 mapach hold-out (sparowane ziarna losowe)



Rysunek 4: Ewaluacja hold-out: średni wynik (± 2 SE) oraz odsetki wygranych i miejsc top-3.

nagrodzie terminalnej nauczyła się „dożywać do środka stawki”, ale nie grać o wygraną — przy punktacji coraz bardziej promującej wyższe miejsca to bardzo znaczące, a estymaty wartości agresywnych trybów w końcówce gry (mało prób, duża wariancja) przegrywają z ekspercką regułą „przy przewadze — walcz”. Wynik PPO potwierdza to samo z drugiej strony: algorytm uczący się na elementarnych akcjach z lokalnej obserwacji kończy poniżej prostej heurystyki reaktywnej (3% wygranych) — ten sam budżet zainwestowany w decyzję strategiczną nad gotowymi wykonawcami daje wynik o 30% lepszy.

4 Wnioski i dalsze prace

Dwuwarstwowa architektura okazała się właściwym wyborem: oba selektory na nowej warstwie wykonawczej wyraźnie pobijają bota z rundy 1, a cechy niezależne od mapy przeniosły się na areny hold-out bez spadku jakości. Do pobicia eksperckiej kaskady selektorowi uczoneму zabrakło przede wszystkim sygnału premiującego wygrane. Dalsze prace: (1) kształtowanie nagrody z wypukłym bonusem za miejsca 1–3; (2) dłuższy trening z wieloma inicjalizacjami i ablacją K ; (3) selektor DQN na cechach ciągłych, bez dyskretyzacji; (4) planowanie MCTS w warstwie wykonawczej trybu *hunt*.